English below

Guida all'uso del progetto "Database dei bronzi capuani"

La presente guida ha lo scopo di illustrare la struttura del progetto e come poter utilizzare le sue componenti per consultare il database, modificarlo o implementarlo. Per poterlo rendere accessibile, il database è ospitato su Neon.tech e distribuito online su Render al link https://capuan-bronzes-db-map.onrender.com/. E' possibile che, una volta avviata la pagina dopo il click sul link, sia necessario aspettare qualche secondo prima che il database e webGIS siano visibili perché i progetti su Render, dopo un lungo periodo di inattività, vengono sospesi (vanno "in sleep") o il cosiddetto deploy si interrompe.

1. Struttura del Progetto

Il progetto è composto da un database PostgreSQL contenente dati archeologici i dati relativi al vasellame attribuito a produzione capuana in epoca romana, una Flask app per la gestione dei dati e delle visualizzazioni, una mappa interattiva realizzata con Folium, e un repository GitHub che ospita il codice. Il progetto è distribuito online tramite il server Render.

2. Funzionamento del Database

Il database è ospitato su **Neon.tech** ed è denominato capuan_bronzes. Ha due tabelle principali:

- archaeological_objects: contiene informazioni testuali e geografiche sugli oggetti archeologici. I campi descrittivi di ogni oggetto archeologico sono:
    - ID assegnato automaticamente dal database,
    - ID unico assegnato dall'utente all'oggetto in ordine sequenziale,
    - la cronologia (solitamente espressa in secoli),
    - la forma,
    - l'ID del luogo di conservazione in ordine sequenziale,
    - il nome e lo stato del luogo di conservazione,
    - le coordinate geografiche del luogo di conservazione,
    - il numero di inventario,
    - l'ID del luogo di ritrovamento dell'oggetto,
    - il nome e la regione del luogo di ritrovamento,
    - le coordinate del luogo di ritrovamento,
    - la fonte bibliografica da cui sono ricavati i dati dell'oggetto,

- le dimensioni,
- la descrizione,
- il luogo di produzione (Capua quando l'attribuzione è sicura, Capua? o Campania quando è incerta),
- la tipologia del vasi,
- i riferimenti bibliografici sulla tipologia,
- una descrizione di anse/manici e base/piede se presenti,
- decorazione se presente,
- le tecniche decorative,
- l'iconografia,
- le tecniche produttive del vaso,
- le analisi archeometriche se sono state effettuate,
- il tipo di analisi, le materie prime e la loro provenienza a seguito dei risultati delle analisi,
- il bollo se presente ed il testo del bollo.

È presente anche un campo in cui poter inserire altre informazioni, se necessarie.

- images: contiene le immagini associate agli oggetti. I campi principali sono:
    - id (generato automaticamente dal database)
    - archaeological_object_id (collegamento alla tabella oggetti)
    - path (nome del file, campo compilato automaticamente)
    - image_type (.jpeg, .jpg, .png, etc. Campo compilato automaticamente)

Sono presenti due colonne geografiche (storing_place_location e finding_spot_location) che contengono geometrie (punti) in formato PostGIS.
Il database è implementabile e possono essere aggiunti nuovi dati.

3. Flask App

La Flask app (`app.py`) gestisce il backend dell'applicazione, cioè il codice scritto in Python del server che gestisce i dati del database e le operazioni ad esso connesse, processa il form con cui inserire i dati nel database e permette di fare l'upload delle immagini nel database. Gestisce inoltre le rotte per la visualizzazione dei dati in una taballe (view_data) e la generazione della mappa. Il backend è sviluppato con Flask e collegato al database PostgreSQL/PostGIS tramite SQLAlchemy, una libreria Python che collega

la flask app al database SQL e gestisce le relazioni tra le due tabelle principali (archaeological_objects e images). I file principali sono:

- app.py: gestisce le rotte dell'applicazione web che permette di utilizzare il database SQL.
- db_utils.py: si occupa delle interazioni della mappa Folium con il database (es. recupero dati).
- gis_utils.py: elabora i dati geografici e genera la mappa Folium.

Si tratta di tre file in cui sono contenuti i codici Python per generare i backend. Se ad esempio si vuole modificare il filtro di ricerca avanzata nel database o fare dei cambiamenti nella gestione dei dati, bisogna utilizzare le backend.
La cartella static/uploads contiene le immagini degli oggetti che vengono caricate sul database e qui vengono salvate in automatico una volta completato l'inserimento nel form. Le immagini vengono referenziate dinamicamente nella mappa e nella tabella.

4. Templates HTML

I file nella cartella `templates` definiscono l'interfaccia utente (o frontend):
- `index.html`: pagina iniziale con link alle altre sezioni (la tabella contenente tutti gli oggetti archeologici inseriti nel database con le relative immagini e la mappa)
- `form.html`: modulo per l'inserimento di nuovi dati.
- `edit_object.html`: pagina per la modifica degli oggetti.
- `map.html`: visualizzazione della mappa interattiva.
- `view_data.html`: visualizzazione tabellare dei dati.

In particolare, la pagina Visualizza dati (view_data.html), accessibile in fondo alla pagina del form di inserimento dei dati, mostra tutti i dati presenti nel database in forma tabellare. È possibile effettuare ricerche tramite la barra di ricerca che filtra i risultati a seconda della ricerca che l'utente vuole fare, anche se la ricerca ha bisogno di essere migliorata. Sarebbe infatti utile il completamento automatico dei campi ed il conteggio dei risultati della ricerca. Le immagini vengono mostrate accanto ai dati e sono ingrandibili con un click del mouse sopra l'immagine. È possibile inoltre aprire le immagini in un'altra finestra del browser con il click destro del mouse e, allo stesso modo, salvarle. La tabella è generata dinamicamente e collegata a una route nella Flask (/view_data) che carica i dati dal database.

5. Mappa Folium

La mappa, accessibile online, è generata dalla backend `gis_utils.py` e mostra i punti geografici organizzati su due livelli, luoghi di conservazione (marker rossi) e luoghi di ritrovamento (marker azzurri). Ogni marker presenta un pop up, che si apre cliccando su di esso, in cui sono presentati in forma tabellare i dati degli oggetti archeologici associati al luogo, comprese le immagini che è possibile ingrandire e salvare con il tasto destro del mouse > apri in un'altra finestra.
Al passaggio del mouse sui marker, inoltre, viene mostrato il nome del luogo di rinvenimento o di conservazione ed il numero di oggetti ad essi associati.
I dati sono estratti dal database tramite la backend `db_utils.py`. Le immagini sono caricate da GitHub al percorso:
https://raw.githubusercontent.com/capuan-bronzes/capuan_bronzes_db_map/main/static/uploads/<nome_file>

La barra di ricerca nella mappa (simile a quella di view_data) può essere attivata con una funzione JavaScript integrata nella mappa o personalizzata direttamente in gis_utils.py, ma migliorabile in quanto sarebbe utile inserire un filtro dei risultati mostrando sulla mappa solo i punti corrispondenti ai risultati della ricerca.

6. Il database su neon.tech

Il database è ospitato su neon.tech da cui è possibile ottenere tutti i parametri di connessione al database cliccando su "Connect". Per ottenere i dati in CSV o Excel, non esiste un pulsante di download su neon.tech ma è possibile seguire tre strade:

1) Usando un client SQL (come pgAdmin): connettersi al database usando la connection string ed eseguire una query come
   SELECT *
   FROM archaeological_objects;
   per esportare in CSV o XLSX
2) Usando un terminale:
   \copy (SELECT * FROM archaeological_objects) TO 'archaeological_objects.csv' WITH CSV HEADER;
3) Tramite Excel connettersi direttamente al database sql in questo modo:

   - Installa il driver ODBC PostgreSQL oppure un add-in Excel specifico
   - In Excel: Dati → Ottieni dati → Da database → Da PostgreSQL
   - Usa la stessa stringa di connessione Neon

7. Repository GitHub

Il repository contiene tutto il progetto con i vari files, inclusa la cartella uploads contenente le immagini caricate sul database, e i codici Python delle backend e html delle frontend. Per utilizzarlo e modificarlo, seguire i seguenti passaggi:

1. Creare un account GitHub (se non ne hai uno).
2. Accedere al repository pubblico: https://github.com/capuan-bronzes/capuan_bronzes_db_map
3. Per modificare:
   - Clonare il progetto: la clonazione del progetto GitHub sul proprio PC è utile per tracciare tutte le modifiche al codice, poter tornare alle versioni precedenti degli elementi contenuti nel repo, ed avere una storia completa delle modifiche. Per clonare bisogna aprire il terminale di Windows o MacOS e digitare
     ```
     git clone  https://github.com/capuan-bronzes/capuan_bronzes_db_map
     cd capuan_bronzes_db_map
     ```
   - Caricare le modifiche sul repository online: sempre tramite il terminale, digitare:
     ```
     git add .
     git commit -m "Descrizione della modifica"
     ```

```
git push origin main
```

Le immagini vengono sincronizzate automaticamente nella cartella static/uploads (in locale) grazie a un workflow GitHub Actions che aggiorna il repository ogni ora. Se ci sono nuovi file in static/uploads, crea automaticamente un commit e push. La sincronizzazione automatica fa sì che il database e la mappa ospitati online su Render, che è connesso al repository GitHub, siano sempre aggiornati. Il workflow viene configurato con la creazione di un GitHub Personal Access Token (PAT), ma l'utente che clona il repository non deve generare un nuovo token per il workflow, a meno che non voglia eseguire il workflow sul proprio account GitHub. Il codice del workflow è salvato nella cartella .github presente nel repository.

7. Zotero

La bibliografia relativa al progetto è disponibile pubblicamente su Zotero (fare il login con le proprie credenziali di Zotero) al link
https://www.zotero.org/groups/5499560/capuan_bronze_production_bibliography/library

Credenziali utili

- GitHub: accedere con le proprie credenziali (solo se necessario fare modifiche al repository)
- Parametri di connessione al database Neon.tech:
  - connection string:
    postgresql://capuan_bronzes_owner:npg_r6HTmaW9XPOg@ep-morning-frost-a2jog6ch-pooler.eu-central-1.aws.neon.tech/capuan_bronzes?sslmode=require&channel_binding=require
  - psql: psql
    'postgresql://capuan_bronzes_owner:npg_r6HTmaW9XPOg@ep-morning-frost-a2jog6ch-pooler.eu-central-1.aws.neon.tech/capuan_bronzes?sslmode=require&channel_binding=require'
  - Parametri (anche per connettere il database al GIS desktop):
    HOST='ep-morning-frost-a2jog6ch-pooler.eu-central-1.aws.neon.tech'
    DATABASE='capuan_bronzes'
    USER='capuan_bronzes_owner'
    PASSWORD='npg_r6HTmaW9XPOg'
    SSLMODE='require'
    CHANNELBINDING='require'

User Guide to the Project "Database of Capuan Bronzes"

This guide aims to illustrate the structure of the project and explain how its components can be used to consult, modify, or expand the database. To make it accessible, the database is hosted on Neon.tech and distributed online via Render at the following link: https://capuan-bronzes-db-map.onrender.com/.

It is possible that, after opening the page by clicking the link, you may need to wait a few seconds before the database and the webGIS become visible. This is because projects on Render, after a long period of inactivity, are suspended (they go into "sleep" mode) or the so-called deploy process stops.

1. Project Structure

The project consists of:

- a PostgreSQL database containing archaeological data related to bronze vessels attributed to Capuan production in the Roman period,
- a Flask application for data management and visualization,
- an interactive map created with Folium,
- and a GitHub repository hosting the source code.

The project is deployed online via the Render server.

2. Database Functionality

To make the database accessible online, it is hosted on Neon.tech and is named **capuan_bronzes**. It includes two main tables:

• archaeological_objects

This table contains textual and geographical information about the archaeological objects.

The descriptive fields include:

- Automatically generated database ID
- User-assigned sequential unique object ID
- Chronology (usually expressed in centuries)
- Form
- Sequential ID of the storing place
- Name and country of the storing place

- Geographic coordinates of the storing place
- Inventory number
- ID of the finding spot
- Name and region of the finding spot
- Geographic coordinates of the finding spot
- Bibliographic source from which the object data are derived
- Dimensions
- Description
- Place of production (Capua when certain; Capua? or Campania when uncertain)
- Vessel typology
- Bibliographic references related to the typology
- Description of handles and base/foot (if present)
- Decoration (if present)
- Decorative techniques
- Iconography
- Production techniques
- Archaeometric analyses (if conducted)
- Type of analysis
- Raw materials and their provenance based on analytical results
- Stamp (if present) and stamp text
- Additional notes field (if necessary)

• images

This table contains the images associated with the objects. The main fields are:

- id (automatically generated by the database)
- archaeological_object_id (foreign key linking to archaeological_objects)
- path (file name, automatically generated)
- image_type (.jpeg, .jpg, .png, etc., automatically generated)

Two geographic columns are also present:

- storing_place_location
- finding_spot_location

These contain point geometries in PostGIS format.

The database is expandable, and new data can be added at any time.

3. Flask Application

The Flask application (app.py) manages the backend of the system, i.e., the Python server-side code that handles database operations and related processes. It processes the data entry form and allows image uploads into the database.

It also manages the routes for:

- tabular data visualization (view_data)
- map generation

The backend is developed with Flask and connected to the PostgreSQL/PostGIS database via SQLAlchemy, a Python library that links the Flask application to the SQL database and manages the relationships between the two main tables (archaeological_objects and images).

The main backend files are:

- **app.py** – manages the web application routes that allow interaction with the SQL database
- **db_utils.py** – handles interactions between the Folium map and the database (e.g., data retrieval)
- **gis_utils.py** – processes geographic data and generates the Folium map

These three files contain the Python code that generates the backend. For example, to modify the advanced search filter or make changes to data management, the backend files must be edited.

The folder static/uploads contains the images of the objects. When images are uploaded through the form, they are automatically saved in this folder. The images are dynamically referenced in both the map and the table view.

4. HTML Templates

The files in the templates folder define the user interface (frontend):

- index.html – homepage with links to other sections (the table of all archaeological objects with images and the map)

- form.html – form for entering new data
- edit_object.html – page for editing objects
- map.html – interactive map view
- view_data.html – tabular data view

In particular, the **View Data** page (view_data.html), accessible at the bottom of the data entry form page, displays all database records in tabular format.

It is possible to perform searches using the search bar, which filters results according to the user's query. However, the search function could be improved. In particular, autocomplete functionality and a search results counter would be useful enhancements.

Images are displayed alongside the data and can be enlarged by clicking on them. They can also be opened in a new browser window or saved via right-click.

The table is dynamically generated and connected to a Flask route (/view_data) that retrieves data from the database.

5. Folium Map

The online map is generated by the backend file gis_utils.py and displays geographic points organized into two layers:

- Storing places (red markers)
- Finding spots (light blue markers)

Each marker includes a pop-up window that opens when clicked. The pop-up displays the associated archaeological objects in tabular form, including images that can be enlarged or saved via right-click → open in a new window.

When hovering over a marker, the name of the storing place or finding spot and the number of associated objects are displayed.

The data are extracted from the database via the backend file db_utils.py. Images are loaded from GitHub at the following path:

https://raw.githubusercontent.com/capuan-bronzes/capuan_bronzes_db_map/main/static/uploads/<file_name>

The search bar on the map (similar to the one in view_data) can be activated through an integrated JavaScript function or customized directly in gis_utils.py. It could be improved by implementing a filter that displays only the points corresponding to search results.

6. The Database on Neon.tech

The database is hosted on Neon.tech. All connection parameters can be obtained by clicking “Connect” in the Neon dashboard.

There is no direct download button on Neon.tech to export data in CSV or Excel format. However, there are three possible methods:

1) Using an SQL client (e.g., pgAdmin)

Connect to the database using the connection string and run a query such as:

```
SELECT *
FROM archaeological_objects;
```

Then export the results in CSV or XLSX format.

2) Using a terminal

```
\copy (SELECT * FROM archaeological_objects) TO 'archaeological_objects.csv' WITH CSV HEADER;
```

3) Connecting directly from Excel

- Install the PostgreSQL ODBC driver or a specific Excel add-in
- In Excel: Data → Get Data → From Database → From PostgreSQL
- Use the same Neon connection string

---

7. GitHub Repository

The repository contains the entire project, including:

- the uploads folder with images stored in the database,
- backend Python code,
- frontend HTML templates.

To use and modify the project:

1. Create a GitHub account (if you do not already have one).
2. Access the public repository:
   https://github.com/capuan-bronzes/capuan_bronzes_db_map

To modify the project:

Clone the repository

Open the Windows or macOS terminal and type:

```
git clone  https://github.com/capuan-bronzes/capuan_bronzes_db_map
cd capuan_bronzes_db_map
```

Cloning allows you to track all code modifications, revert to previous versions, and maintain a complete history of changes.

Upload modifications to the online repository

```
git add .
git commit -m "Description of the modification"
git push origin main
```

Images are automatically synchronized in the static/uploads folder (locally) through a GitHub Actions workflow that updates the repository every hour. If new files are detected in static/uploads, the workflow automatically commits and pushes them.

This automatic synchronization ensures that the database and map hosted online on Render, which is connected to the GitHub repository, remain up to date.

The workflow is configured using a GitHub Personal Access Token (PAT). Users who clone the repository do not need to generate a new token unless they wish to run the workflow under their own GitHub account.

The workflow code is stored in the .github folder within the repository.

---

8. Zotero

The bibliography related to the project is publicly available on Zotero (login required with personal Zotero credentials) at
https://www.zotero.org/groups/5499560/capuan_bronze_production_bibliography/library

Useful credentials

- GitHub: use your own account credentials (only if you need to modify the repository)
- Neon.tech connection parameters:
  - connection string: postgresql://capuan_bronzes_owner:npg_r6HTmaW9XPOg@ep-morning-frost-a2jog6ch-pooler.eu-central-1.aws.neon.tech/capuan_bronzes?sslmode=require&channel_binding=require
  - psql: psql 'postgresql://capuan_bronzes_owner:npg_r6HTmaW9XPOg@ep-morning-frost-a2jog6ch-pooler.eu-central-1.aws.neon.tech/capuan_bronzes?sslmode=require&channel_binding=require'

- Parameters to connect the GIS desktop app to the database:
  HOST='ep-morning-frost-a2jog6ch-pooler.eu-central-1.aws.neon.tech'
  DATABASE='capuan_bronzes'
  USER='capuan_bronzes_owner'
  PASSWORD='npg_r6HTmaW9XPOg'
  SSLMODE='require'
  CHANNELBINDING='require'